

SWIFT SHIP TRACKER

Salesforce Implementation Report

OVERVIEW

SwiftShip Tracker is a centralized, Salesforce-based parcel management solution. It lets customers book parcels, track delivery status and manage shipments through a Salesforce interface and a conversational AI assistant. Parcel, delivery, sender and receiver information is held in custom objects connected by relationships, so that a parcel can always be traced to its sender, its receiver and its delivery record.

Parcel booking creates a `Parcel__c` record with an auto-numbered Parcel ID. Shipment progress is carried by the `Status__c` picklist (Booked, In Transit, Out for Delivery, Delivered), and delivery tracking is carried by the `Delivery__c` object, which holds the current location as a Geolocation field and an estimated delivery date. Delivery agents update parcel and delivery information through Salesforce. The design adds Email Alerts and Email Templates for customer notifications, Salesforce Flows and Apex for automation, Experience Cloud for customer access, role-based security through profiles, permission sets, a role hierarchy, sharing rules and field-level security, and reports and dashboards for operational visibility. The part that was built and tested end to end is conversational tracking: AgentForce AI, Prompt Builder and an Auto-Launched Flow retrieve a parcel by its ID and return a formatted tracking update.

The system supports the complete parcel lifecycle:



Figure 1. Parcel lifecycle

In this report, a detail is described as documented when the project source configures or evidences it, as design when the source states it as a plan without configuration steps, and as not specified when the source gives no information. No detail in the last category has been invented.

PHASE 1: REQUIREMENT ANALYSIS & PLANNING

1.1 Functional Scope

The functional scope covers parcel booking and record creation; status tracking through `Status__c`; weight management through `Weight__c`; estimated delivery date management through `Estimated_Delivery_Date__c` on both Parcel and Delivery; sender management through `Sender__c`, which stores address, contact number and email and links to many parcels; receiver management through `Receiver__c`, which stores address, contact and email for delivery reference and confirmation; and delivery and current-location tracking through `Delivery__c`. It also covers agent status updates, automated notifications, AI-powered tracking, reports and dashboards, and customer access through Experience Cloud. The stated project scope is a scalable CRM solution for parcel delivery businesses that supports the lifecycle from booking through dispatch and tracking to delivery confirmation.

1.2 Problem Statement

Five problem statements drive the project. A customer or sender trying to book a parcel easily finds that booking needs support assistance or several applications, because parcel operations are not centralized, which leaves them frustrated. A customer or sender trying to track a parcel has no simple real-time tracking experience because shipment information is hard to access, which leaves them confused. A delivery agent trying to update parcel status faces manual handling because there is no centralized operational tracking, which leaves them concerned. A customer trying to receive delivery updates may not get notifications at every stage because communication is not automated, which leaves them uncertain. An administrator trying to monitor parcel operations finds parcel flow and delivery performance hard to see because operational data lacks centralized reporting, which leaves them concerned.

Four alternatives were considered: manual registers, spreadsheet tracking, a standalone parcel portal, and Salesforce CRM with AgentForce AI. Salesforce was selected for centralized parcel management, Flow automation, real-time tracking visibility, conversational AI assistance, secure role-based access, and reports and

dashboards. In the solution mapping, centralized booking answers the booking problem, AgentForce AI with Prompt Builder gives self-service tracking, Flows automate manual status updates, Email Alerts close communication gaps, the four objects structure scattered shipment data, reports and dashboards improve monitoring, and profiles, roles and permission sets secure access.

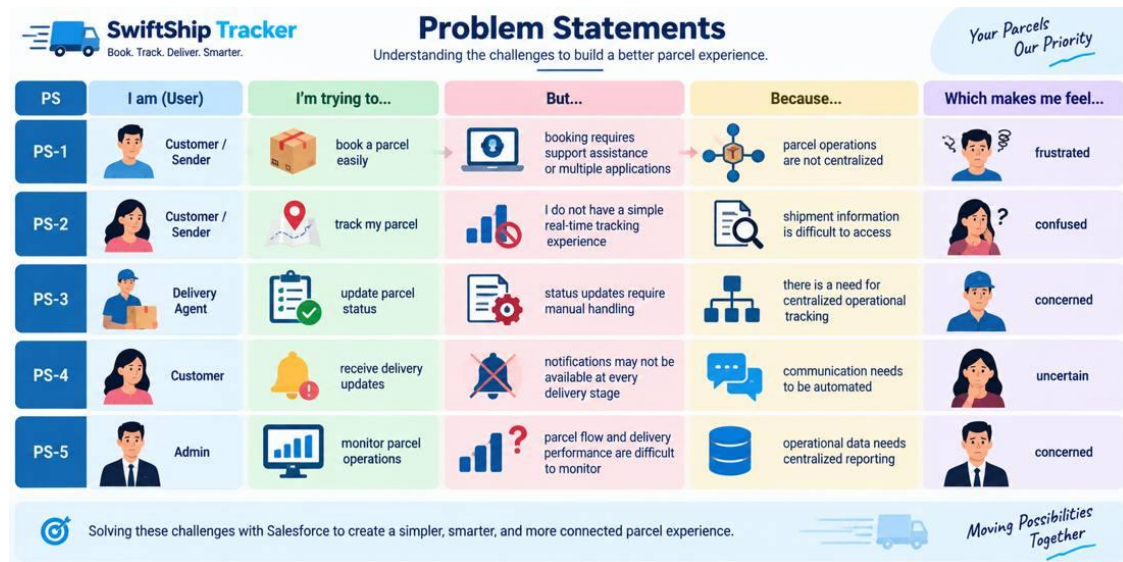


Figure 2. Problem statements PS-1 to PS-5

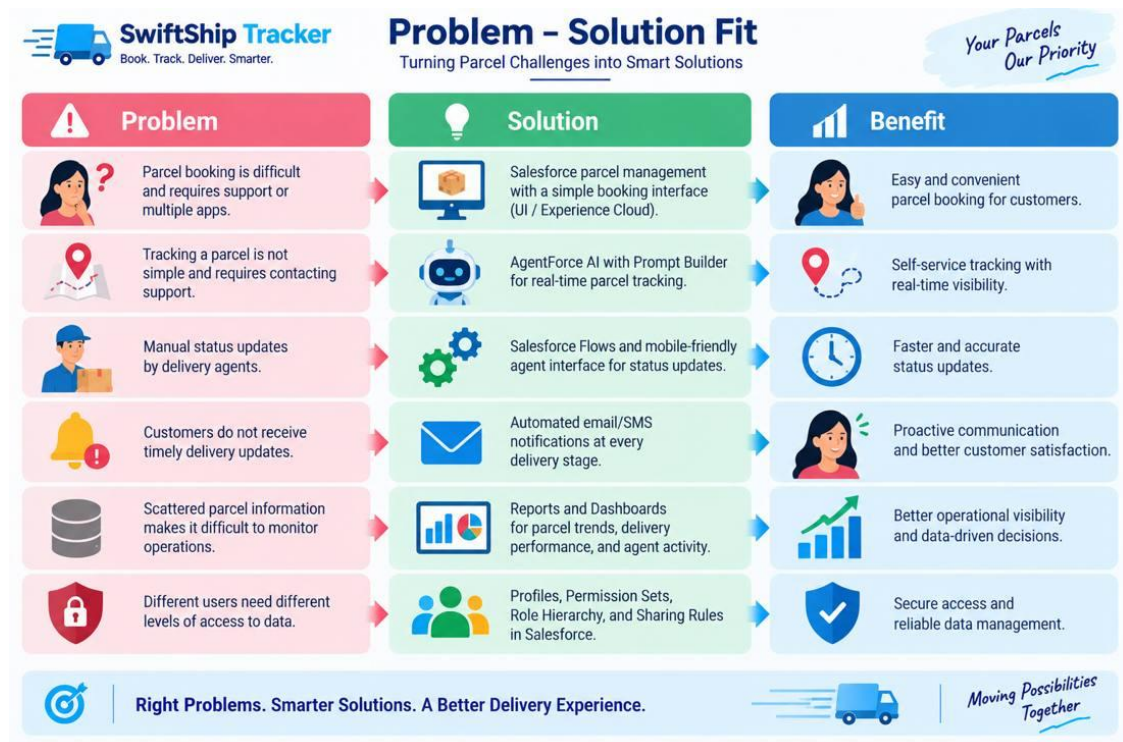


Figure 3. Problem-solution fit

1.3 Stakeholder Mapping

Customers and senders book parcels, track deliveries and receive updates through the portal or the AI agent, providing the sender and parcel details that Salesforce stores. Receivers are stored in `Receiver__c`, and their address, contact and email are used for delivery reference and completion notifications. Delivery agents manage assigned parcels, update status and location, and hand parcels over, using Read and Edit access to Parcel, Delivery, Sender and Receiver. The customer support team handles delivery issues and customer queries using Cases and Parcel details. The admin team configures the org, manages roles and security, and monitors operational performance. The system integrator or developer implements the Flows, Apex, Prompt Builder templates and AgentForce configuration.

1.4 User Personas

The customer or sender wants to book and track parcels without repeatedly contacting support. They book through a portal or app, check status, ask for updates and share tracking information with recipients. Their pain points are delays, unclear tracking and dependence on support or several apps, and they expect real-time visibility, self-service tracking and timely notifications.

The delivery agent wants to update parcel status and delivery information accurately. They manage assigned parcels and move status from Booked to In Transit, Out for Delivery and Delivered. Their pain point is manual status handling without centralized tracking, and they expect one interface, guided Flows and dashboards for quick updates.

Support staff want to resolve delivery issues and answer customer queries, working with Cases and Parcel details. Routine tracking questions are an inferred pain point, drawn from the self-service objective, and they expect AgentForce to reduce those questions. The administrator wants to monitor parcel flow and delivery performance and control access. They configure the org, manage roles and security and review reports and dashboards, and they expect parcel trends, delivery performance and agent activity to be visible in one place.

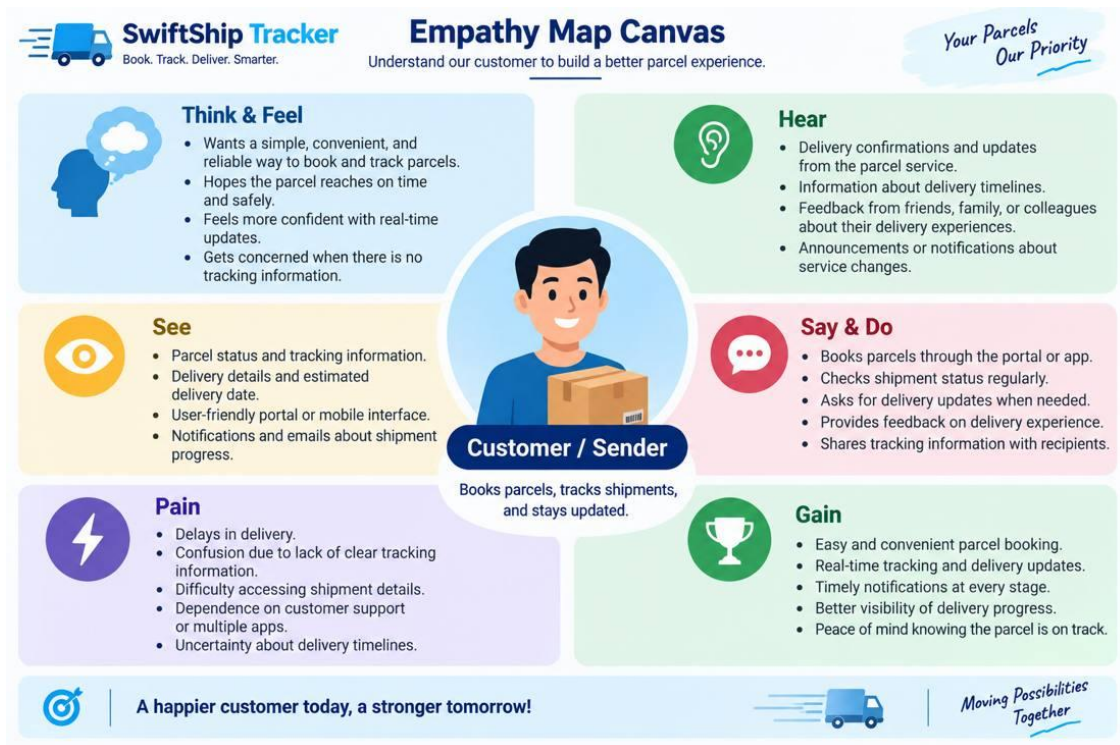


Figure 4. Empathy map for the Customer / Sender persona

1.5 Customer Journey

The journey runs from parcel inquiry, where the customer checks details and options and feels curious, to booking, where providing sender, receiver and parcel details feels easy. The system then creates the parcel record, the parcel is dispatched, and in-transit tracking reassures the customer. When the parcel is out for delivery the customer anticipates it, delivery confirmation delights them, notifications keep them informed, and captured data supports reporting. The data flow is:



The AI tracking flow is:

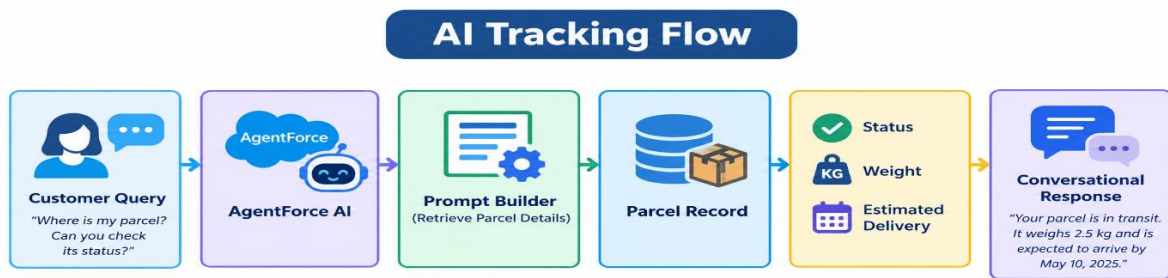


Figure 5. Customer journey map from parcel inquiry to reporting

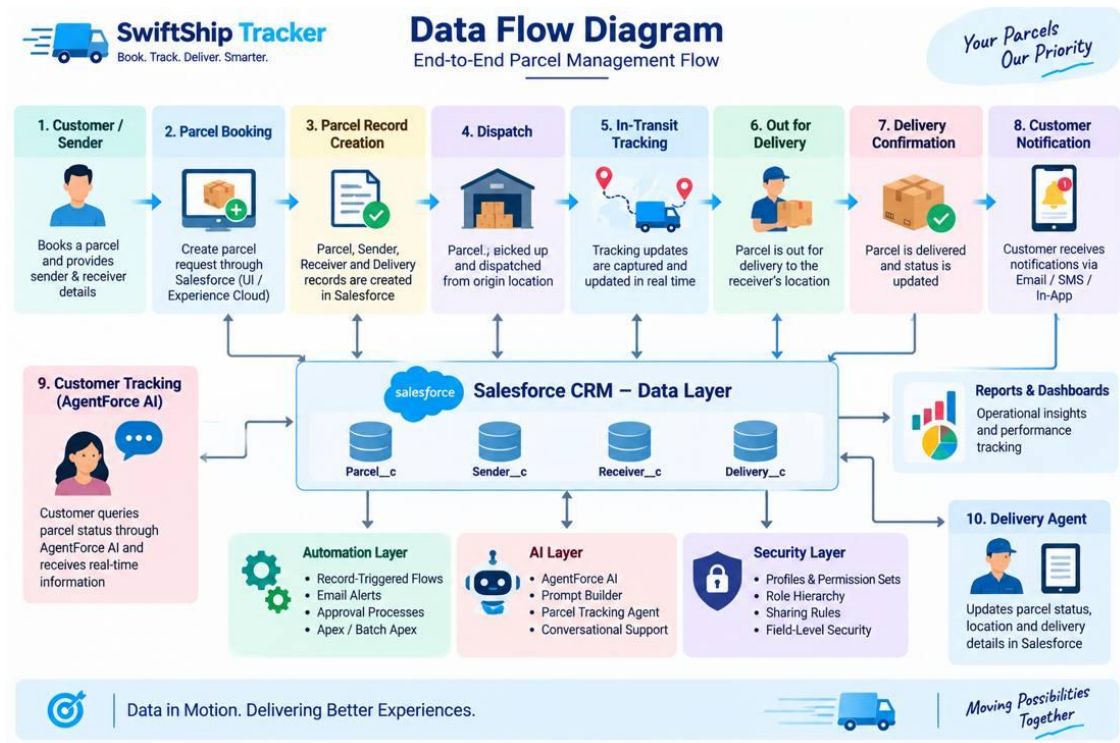


Figure 6. End-to-end data flow diagram

1.6 Functional Requirements

The twelve functional requirements are: FR-1 parcel booking and record creation, realized by `Parcel__c` with Parcel ID format `P-{000}`; FR-2 parcel status tracking through `Status__c`; FR-3 weight management through `Weight__c`; FR-4 estimated delivery date management through `Estimated_Delivery_Date__c`; FR-5 sender information management through `Sender__c`; FR-6 receiver information management through `Receiver__c`; FR-7 delivery and current-location tracking through `Delivery__c` and `Current_Location__c`; FR-8 agent parcel status updates through the Delivery Agent profile; FR-9 automated customer notifications through Email Alerts and Flows (design); FR-10 AI-powered parcel tracking through AgentForce AI, Prompt Builder and a Flow; FR-11 reports and dashboards (design); and FR-12 customer access through Experience Cloud (design).

1.7 Non-Functional Requirements

Usability requires an intuitive Salesforce and customer-facing interface, addressed by custom tabs, a Lightning App, Lightning Record Pages and conversational

AI. Security requires role-based and field-level control, addressed by profiles, permission sets, the role hierarchy, sharing rules and field-level security. Reliability requires accurate status and delivery information, supported by a controlled status picklist and relationship-based data. Performance requires timely Flow and AgentForce responses; the design uses a targeted Get Records filter on Parcel ID, though no response times were measured. Availability comes from cloud-based Salesforce access. Scalability depends on the platform's capacity and on design plans for Experience Cloud and logistics integration. Data integrity relies on controlled access and Lookup relationships, and maintainability on declarative Flows, prompts, topics and configuration that can be updated.

1.8 Technology Stack

The platform is Salesforce Developer Edition. The user interface uses a Lightning App Page, Lightning Record Pages with Dynamic Forms, and Experience Cloud. Data is held in the custom objects `Parcel__c`, `Delivery__c`, `Sender__c` and `Receiver__c`, with standard objects Account, Contact, Case, Task, EmailMessage and User. Business logic uses Salesforce Flows and Apex Classes, and automation uses Record-Triggered, Auto-Launched and Scheduled Flows plus Approval Processes. Batch Apex handles periodic processing, Email Alerts and Email Templates handle notifications, and AgentForce AI with Prompt Builder provides the AI layer. Security uses profiles, permission sets, the role hierarchy, field-level security and sharing rules, and reporting uses reports and dashboards. Google Sheets and Docs supported initial data planning and documentation. Of these, the Developer org, custom objects, Auto-Launched Flow, Prompt Builder, AgentForce and the Swift Ship permission set are configured step by step in the source; the rest are design.

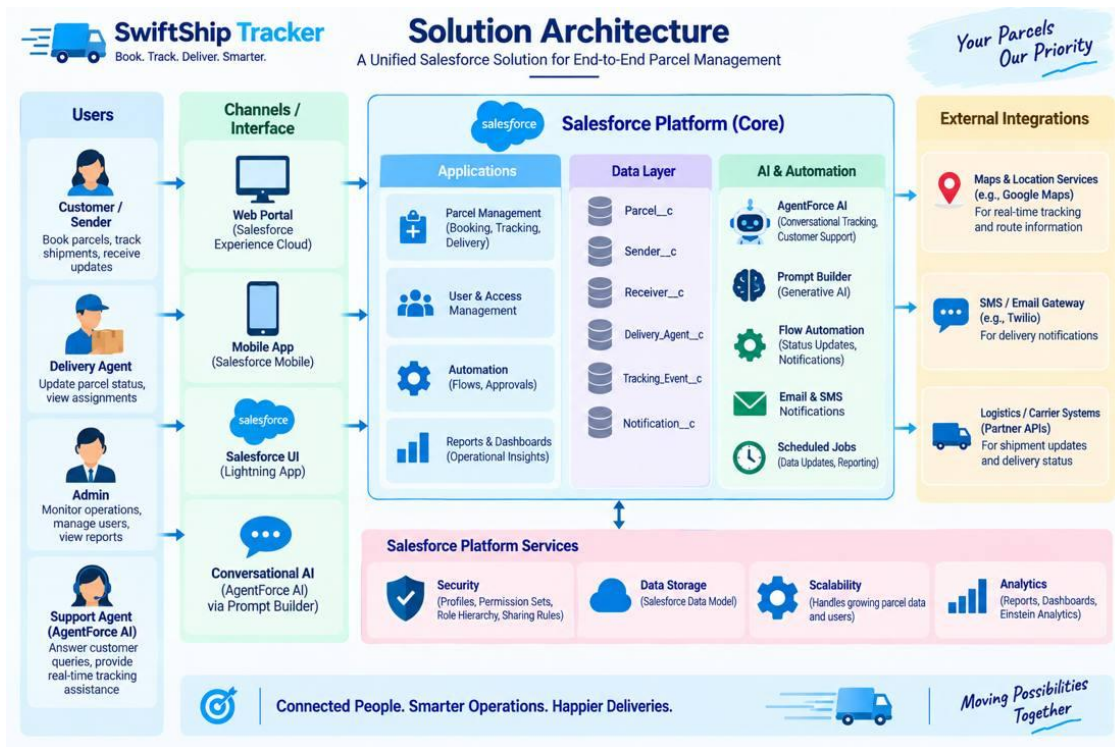


Figure 7. Solution architecture

1.9 Project Planning & Sprint Schedule

The project followed Agile methodology with Epics, Stories and Story Points, sprint-based execution and velocity-based estimation. Sprint 1 (Developer Setup, 3 points) covered creating and configuring the Salesforce environment. Sprint 2 (Data Modeling, 10 points) covered the four objects with relationships and the dedicated tabs and Lightning App. Sprint 3 (Automation, 6 points) covered validation of parcel details, mandatory fields and booking inputs, and Flows to update status and delivery information. Sprint 4 (Apex, 10 points) covered Apex triggers and classes and Batch Apex jobs. Sprint 5 (Reports, 4 points) covered reports on parcel status, delivery performance and operational metrics, and Sprint 6 (Dashboards and AI, 4 points) covered dashboards and AgentForce tracking assistance. Each sprint is planned at 6 days and 20 points; Sprint 1 ran from 1 to 2 October 2026 with 20 points completed. No completion data is recorded for the other sprints, and the story points listed do not sum to the planned 20 per sprint, so no velocity figure is derived.

PHASE 2: BACKEND DEVELOPMENT & CONFIGURATIONS

2.0 Environment Setup

The Developer Edition org was created at developer.salesforce.com/signup by entering name, email, job title (Developer), company (college name) and country (India). The verification email, which can take from 10 minutes to 2 hours, is opened and Verify Account selected, then a password and security question are set, after which the user lands on the Setup home page. The org includes Agentforce and Data Cloud capabilities.

2.1 Data Schema & Relationships

Parcel__c is the core parcel record. Its fields are Parcel ID (Parcel_ID__c, Auto Number P-{000}), Status (Status__c, Picklist with Booked, In Transit, Out for Delivery and Delivered), Weight (Weight__c, Number), Estimated Delivery (Estimated_Delivery_Date__c, Date) and Sender (Sender__c, Lookup). The source describes Parcel as linked to Sender and Receiver and connected to Delivery, but its field table lists only the Sender lookup; the association with Receiver is held by the lookup on Receiver__c.

Delivery__c is the delivery tracking record, with Current Location (Current_Location__c, Geolocation holding longitude and latitude), Estimated Delivery (Estimated_Delivery_Date__c, Date), Sender (Sender__c, Lookup) and Parcel (Parcel__c, Lookup). Sender__c stores the sender profile, with Sender Address (Sender_Adress__c, Geolocation), Sender Contact (Sende_Contact__c, Phone) and Sender Email (Sender_Email__c, Email), and uses Sender Name as its record name. Receiver__c stores the receiver profile, with Receiver Address (Receiver_Adress__c, Geolocation), Receiver Contact (Receiver_Contact__c, Phone), Receiver Email (Receiver_Email__c, Email), a Sender lookup and a Parcel lookup, and uses Receiver Name as its record name.

The API names above are as written in the source's field tables. Its permission-set screenshots show different spellings, such as `Sender_Address__c`, `Sender_Contact__c`, `Receiver_s_Address__c`, `Receiver_s_Contact__c` and `Receivers_Email__c`, plus a `Sender_details__c` field on `Delivery` and a `Reciever__c` lookup on `Receiver`. The final API names should be verified in Object Manager before they are used in Flows, Apex or reports.

The planned standard objects are `Account` and `Contact` for customer context, `Case` for delivery-related issues handled by support, `Task` for operational follow-up, `EmailMessage` for customer email records, and `User` for agents, managers, administrators and the Einstein Service Agent user. The source lists these without configuration details.

2.2 Relational Model

All relationships documented in the source are Lookups. `Parcel__c` looks up `Sender__c`, giving sender details for a parcel and a sender-to-parcels history. `Delivery__c` looks up both `Parcel__c` and `Sender__c`, linking real-time location and status to a parcel and its sender. `Receiver__c` looks up both `Parcel__c` and `Sender__c`, tying the receiver to the shipment and associating both ends of it. One sender can therefore have many parcels. Master-Detail relationships are not used or specified.

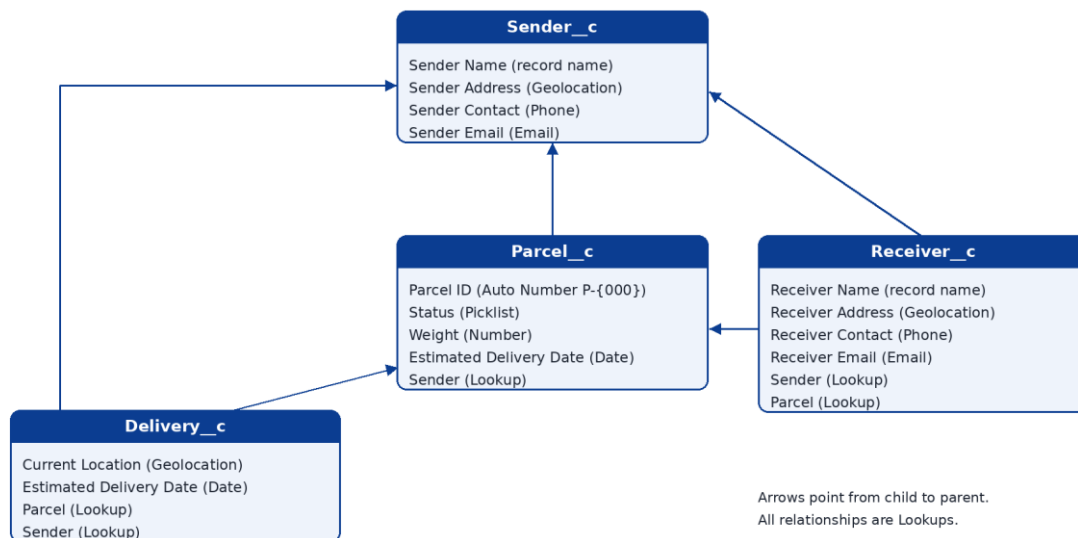


Figure 8. Data model: four custom objects and their Lookup relationships

2.3 Object Configuration

Each custom object is created from Setup, Object Manager, Create, Custom Object. The labels are Parcel, Delivery, Sender and Receiver, with plural labels Parcels, Deliveries, Sender's and Receiver's, and text record names Parcel Name, DeliveryName, SenderName and ReceiverName. For all four objects Allow Reports, Allow Activities, Track Field History and Allow Search are enabled. Allow Sharing, Allow Bulk API Access and Allow Streaming API Access are selected, the deployment status is Deployed, and the Notes and Attachments related list is added to the default page layout.

Custom Object Tabs are then created from Setup, Tabs, New for each object. A tab style is chosen, the Add to Profiles settings are left at default, Include Tab is unchecked on the Add to Custom Apps page, and the option to append the tab to users' existing personal customizations stays checked.

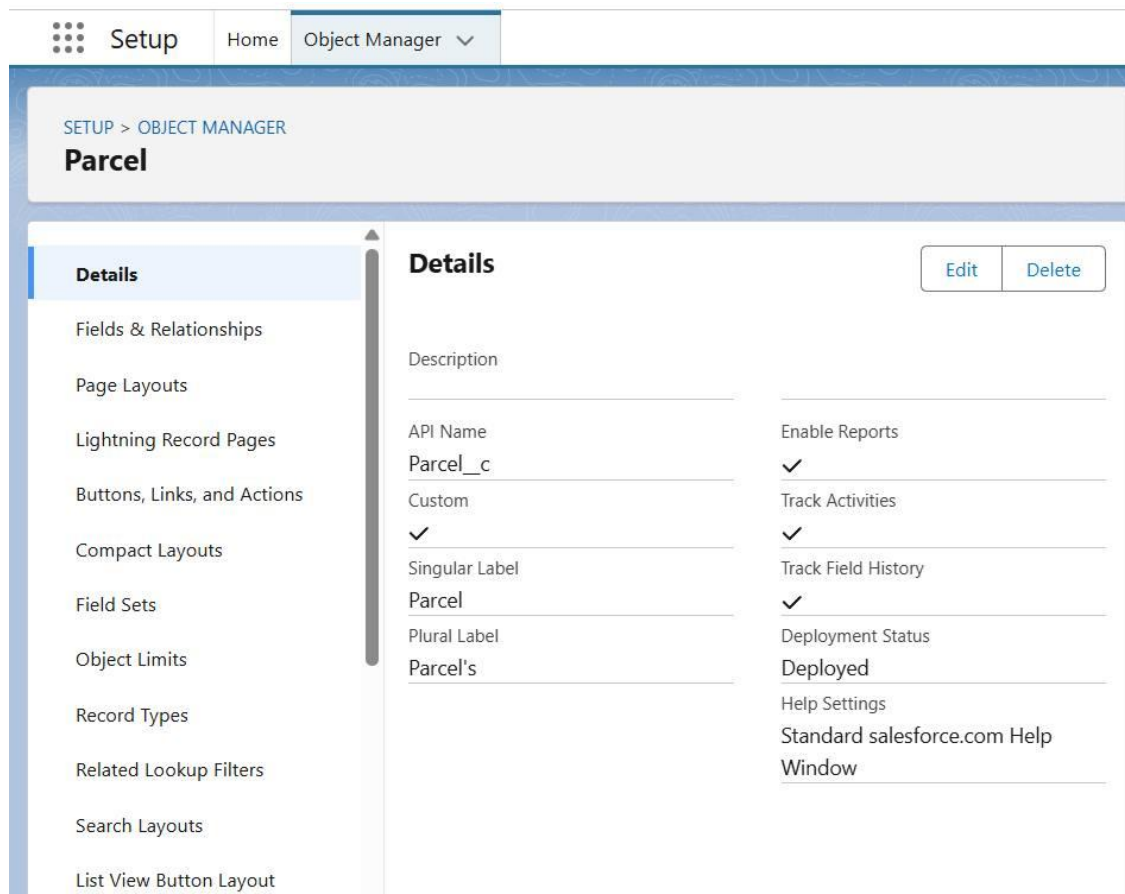


Figure 9. Parcel custom object created in Object Manager

2.4 Fields & Relationships

Fields are created from Object Manager, the object, Fields & Relationships, New, then Next, Next and Save & New. Parcel ID uses data type Auto Number with display format P-{000} and starting number 1. Current Location, Sender Address and Receiver Address use the Geolocation data type with Latitude/Longitude display notation and 15 decimal places. Status is a picklist with the four lifecycle values, Weight is a Number, and Estimated Delivery is a Date. Contact fields use the Phone type, email fields use the Email type, and Sender and Parcel relationships are Lookup fields on the child objects. Geolocation fields store coordinates rather than street addresses, and the source gives no geocoding or map configuration.

2.5 Data Integrity

Mandatory-field and booking-input validation is planned in sprint story USN-4, but no validation rule formulas are specified. Relationship integrity comes from the

Lookup relationships among the four objects. Parcel data is controlled by the fixed status picklist and the auto-numbered Parcel ID, and status accuracy depends on the defined lifecycle and on agents updating status through Flows and Edit access. Track Field History is enabled on all four objects, which gives an audit trail. Access to sensitive fields is restricted by design to the Admin and Agent profiles.

2.6 Business Automation: Salesforce Flows

An Auto-Launched Flow retrieves a parcel by Parcel ID, passes it to the Prompt Builder action and returns the generated tracking text to AgentForce. The Flow is named Parcel Details and has two variables: `ids`, a Text variable available for input, and `output`, a Text variable available for output. Its sequence is Start, Get Records, Action, Assignment, End. The Get Records element (`get parcel records`) selects the Parcel record whose Parcel ID equals `ids`. The Action element (`Parcel Updates`) invokes the Retrieve Parcel Details prompt template with `objectToSummarize` set to the parcel from Get Records. The Assignment element sets `output` to the action's Prompt Response.

```
INPUT  ids  (Text)
parcel  = GET Parcel__c WHERE Parcel_ID__c = ids
response = INVOKE Prompt Template
"Retrieve_Parcel_Details" WITH objectToSummarize =
parcel
output  = response.PromptResponse
RETURN output
```

After saving and activating the Flow, Edit Access is opened from the Flows list. The option to override the default behavior and restrict access to enabled profiles is selected, and the required profiles are moved to Enabled Profiles; the source screenshot shows Einstein Agent User, External Apps Login User, External Einstein Agent User and External Identity User. The source recommends running the Flow in System Context Without Sharing if data access is restricted. Record-Triggered Flows,

Scheduled Flows and Approval Processes are planned, but their trigger criteria and email templates are not specified.

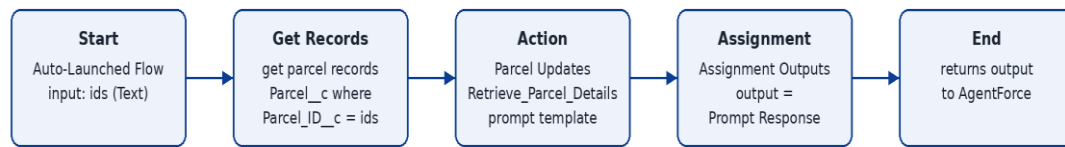


Figure 10. Parcel Details Auto-Launched Flow sequence

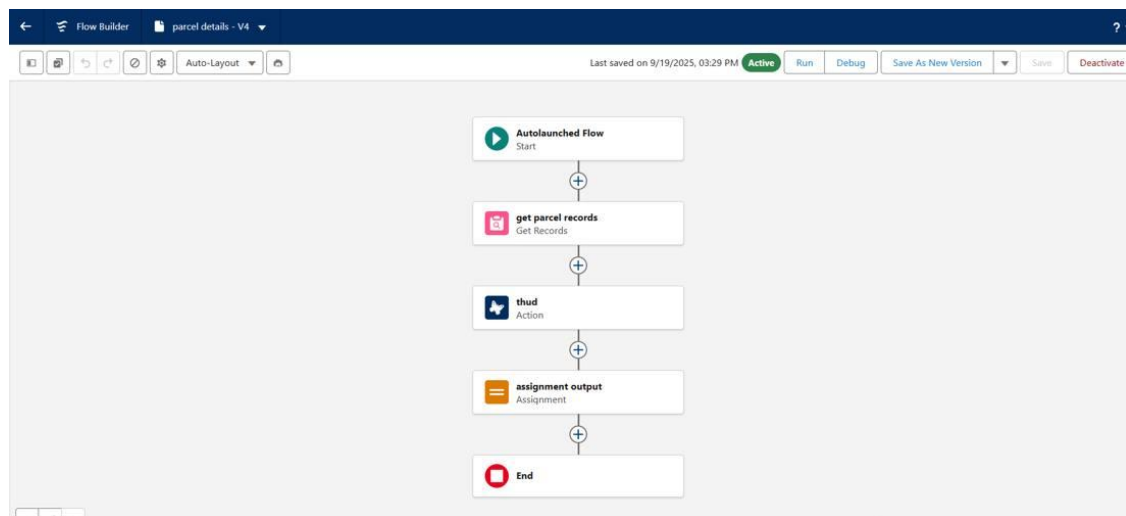


Figure 11. Parcel Details Flow in Flow Builder

2.7 Prompt Builder

The prompt template is a Record Summary template on the Parcel object (Parcel__c), started from scratch. Its label is Retrieve Parcel Details, its API name is Retrieve_Parcel_Details, and its description states that it retrieves parcel details where Parcel_ID matches the user input. The input is objectToSummarize (Parcel__c), and the preview screenshot shows the Standard model type with OpenAI GPT 4 Omni Mini. The template text is:

Retrieve details from Parcel__c where Parcel_ID__c matches user input.

Return response in this format:



Parcel Tracking Update

- Parcel Name: {!\$Input:Parcel__c.Name}
- Parcel ID: {!\$Input:Parcel__c.Parcel_ID__c}
- Status: {!\$Input:Parcel__c.Status__c}
- Weight: {!\$Input:Parcel__c.Weight__c}
- Estimated Delivery Date:
{!\$Input:Parcel__c.Estimated_Delivery_Date__c}

Resource mapping inserts the Parcel object resource and maps the five merge fields to Name, Parcel_ID__c, Status__c, Weight__c and Estimated_Delivery_Date__c. The source screenshot shows a "kg" unit after the weight merge field, which the written template omits, and the weight unit is not defined at field level. The template is saved and activated so that the agent or Flow can use it. In the Prompt Builder preview, the record named Sanjana resolved to Parcel ID P-123, status Booked, weight 10 and estimated delivery date 09/29/2025. The source's suggested agent test query is "Track parcel P-12345", used as a sample Parcel ID.

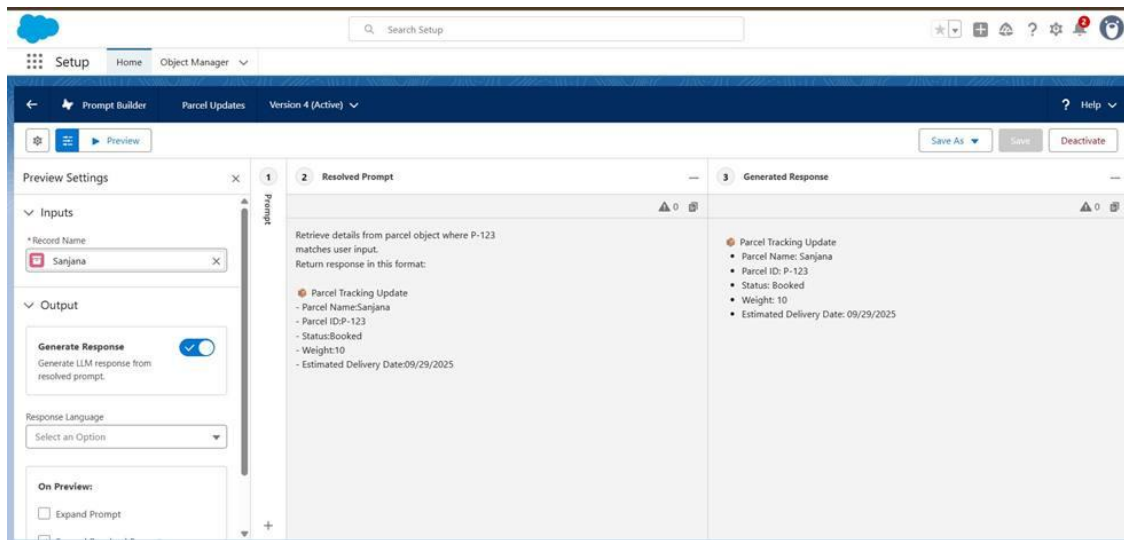


Figure 12. Prompt Builder preview resolving the Retrieve Parcel Details template for a Parcel record

2.8 AgentForce AI

Org setup has three parts. The default agent is enabled from Salesforce Go by searching for Default agent, selecting Get Started and turning it on. In Einstein Setup, Turn On Einstein, Global Languages and Deploy Prompt Templates are switched on. In Agentforce Agents, Agentforce and the Agentforce Default Agent are enabled.

The agent is built by opening the default agent (type Employee) in Builder and renaming it SwiftShip Tracker in Settings. A subagent is created with the description that the agent should give the parcel details related to the user by asking for the parcel ID, which produces the Parcel Details Inquiry subagent. Its classification description says it handles requests to retrieve parcel details by asking for the parcel ID, and its scope limits it to obtaining the parcel ID and retrieving the relevant information. In the Agentforce Asset Library, a new Agent Action is created with reference action type Flow and reference action `parcel details`; its description says that when the user asks for parcel details the agent shares them by matching the parcel ID. The action is then assigned to the subagent through This Subagent's Actions, Add from Asset Library. On activation the checklist warned of overlapping subagents, conflicting instructions and Data Cloud not being enabled, and the source selects Ignore & Activate.

In effect, the Auto-Launched Flow `Parcel Details` is referenced as a Flow-type Agent Action in the Asset Library, added to the Parcel Details Inquiry subagent, and made available through the SwiftShip Tracker agent. In the first agent preview, the user asked for parcel details, the agent asked for the parcel ID, and after the user answered P-001 it selected the Parcel Details topic, ran the action and returned the tracking update. The Einstein Service Agent user acts as the agent's identity and needs the Swift Ship permission set described in section 4.3.

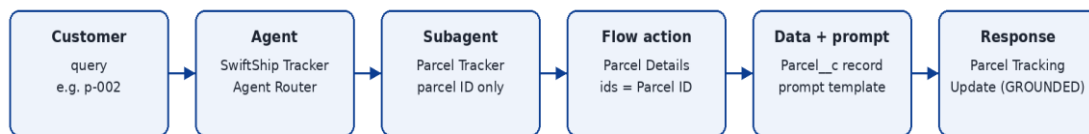


Figure 13. Conversational tracking path from customer query to tracking response

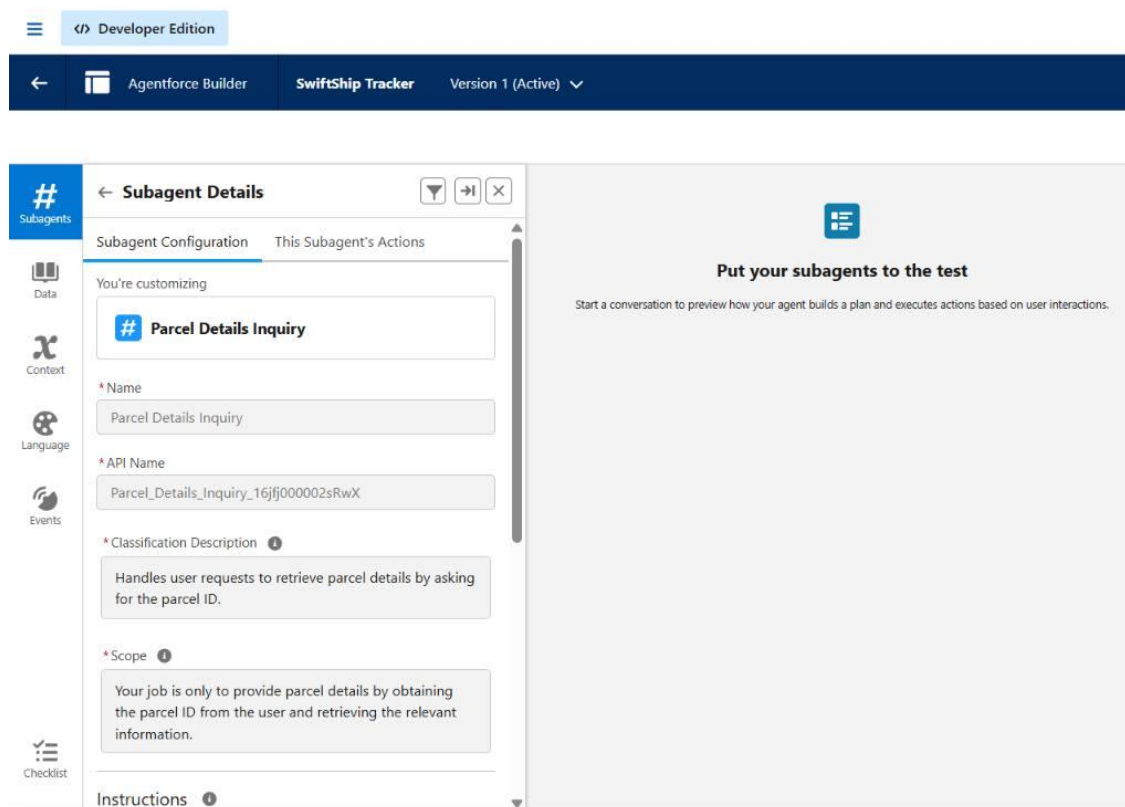


Figure 14. Parcel Details Inquiry subagent configuration with classification description and scope

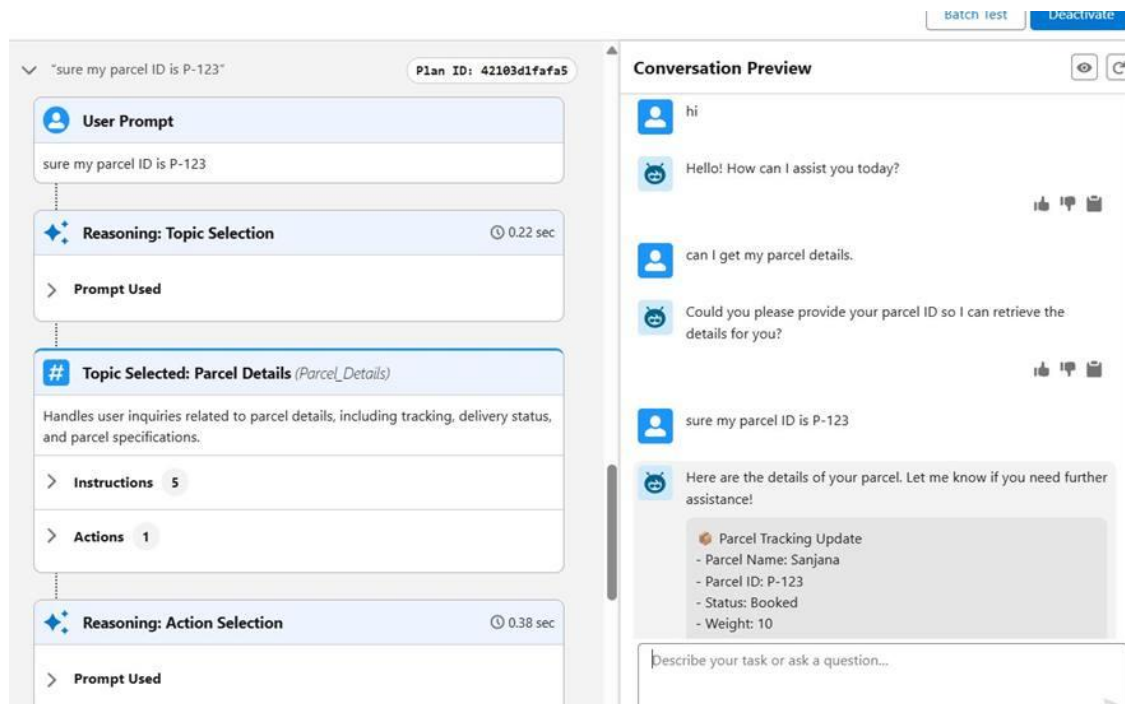


Figure 15. Early agent preview

2.8.1 Final Output Page

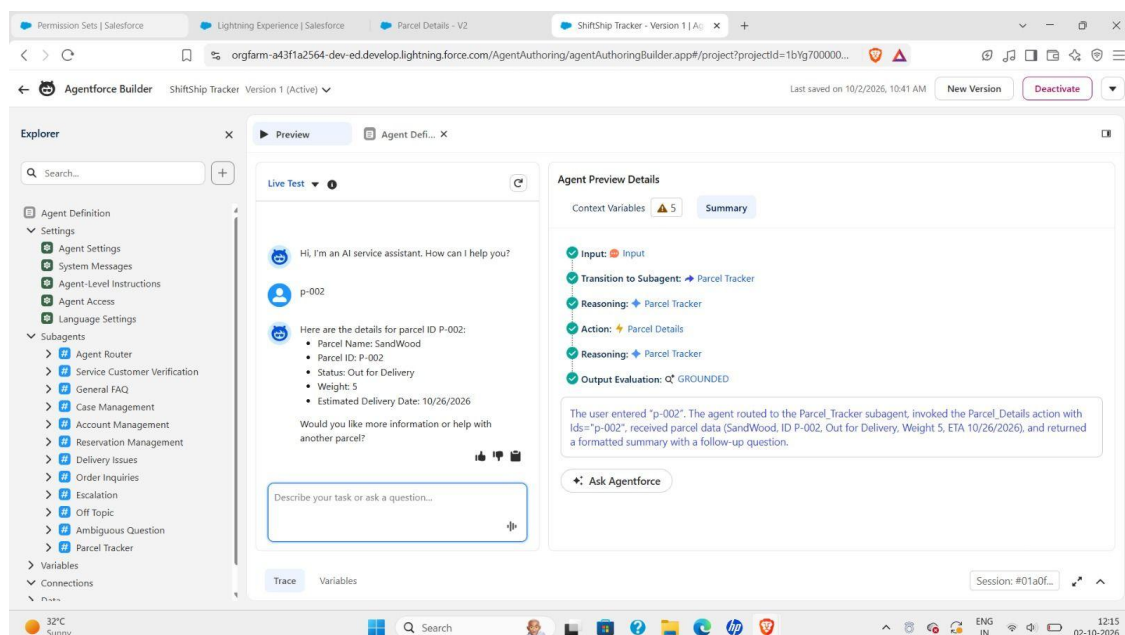


Figure 16. Final output page

In the final run the Builder header shows the agent, spelled "ShiftShip Tracker" on screen, at Version 1 (Active), tested in Live Test. The agent greets the user with "Hi, I'm an AI service assistant. How can I help you?" and the user enters p-002. The agent replies with Parcel Name SandWood, Parcel ID P-002, Status Out for Delivery, Weight

5 and Estimated Delivery Date 10/26/2026, then asks whether the user would like more information or help with another parcel. The output evaluation is GROUNDED.

The trace summary shows the path: Input, Transition to Subagent Parcel Tracker, Reasoning, Action Parcel Details, Reasoning, and Output Evaluation GROUNDED. It states that the agent invoked the `Parcel_Details` action with `Ids="p-002"`, received the parcel data and returned a formatted summary with a follow-up question.

This run demonstrates the full conversational chain from user input through the Agent Router and Parcel Tracker subagent to the Parcel Details Flow action, the `Parcel__c` record and the formatted response. Earlier tests returned P-123 with status Booked, whereas this run returns a different parcel at a later lifecycle stage, so different records and `Status__c` values are retrieved correctly. The lowercase input `p-002` resolved to `P-002`, which the source does not explain but which was observed in this run. The reply lists the same five items defined in the prompt template.

The Builder screen also shows the Agent Definition listing the standard default subagents (Agent Router, Service Customer Verification, General FAQ, Case Management, Account Management, Reservation Management, Delivery Issues, Order Inquiries, Escalation, Off Topic and Ambiguous Question) alongside Parcel Tracker, and the earlier activation checklist warned about overlapping subagents. Context Variables shows a warning count of 5 that the source does not explain. This screenshot names the parcel subagent Parcel Tracker whereas the earlier build screenshots show Parcel Details Inquiry, and a browser tab titled Parcel Details - V2 suggests a second Flow version in this org, though version details are not shown.

2.9 Apex

The source lists Apex Classes, Apex triggers and Batch Apex in the technology stack, planned features, appendix and sprint plan, but provides no class names, trigger names, source code or screenshots, so only the intended roles are described. Apex Classes and triggers are meant to perform critical parcel and delivery actions and automated parcel operations, for example Lead-to-Parcel mapping and delivery notifications. Batch Apex is meant to process and update parcel or delivery records and

to handle periodic updates or notifications. The planned areas are automated parcel operations, delivery processing, periodic processing, notifications and operational maintenance. Before implementation the team should define trigger objects, execution contexts, batch scheduling and Apex test coverage.

PHASE 3: UI/UX CUSTOMIZATION & ANALYTICS

3.1 Lightning App

A Lightning App, titled Parcel Tracker in one screenshot and named SwiftShip Tracker App in the architecture, is shown with navigation tabs including Receiver's, Sender's, Parcel's and Deliveries, and tabs for all four custom objects were created. Reports and Dashboards are part of the design, but report names and dashboard components are not specified, and other modules are not specified. The Setup tab list in the source also shows tabs for unrelated objects in the org, such as Article's and Purchase's, which are not part of the SwiftShip data model.

3.2 Lightning Record Pages

Dynamic Forms for parcel details and Lightning Record Pages are listed as planned UI features, but page layouts and field placement are not specified. The source does include a Receiver record screenshot showing the Details and Related tabs, an Activity timeline and the receiver, email, contact, address, parcel and audit fields. As a proposal, not documented implementation, a Parcel page could show a highlights panel with Parcel ID, Status and Estimated Delivery Date, a Delivery related list, and Dynamic Forms that expose agent-editable fields only to delivery roles.

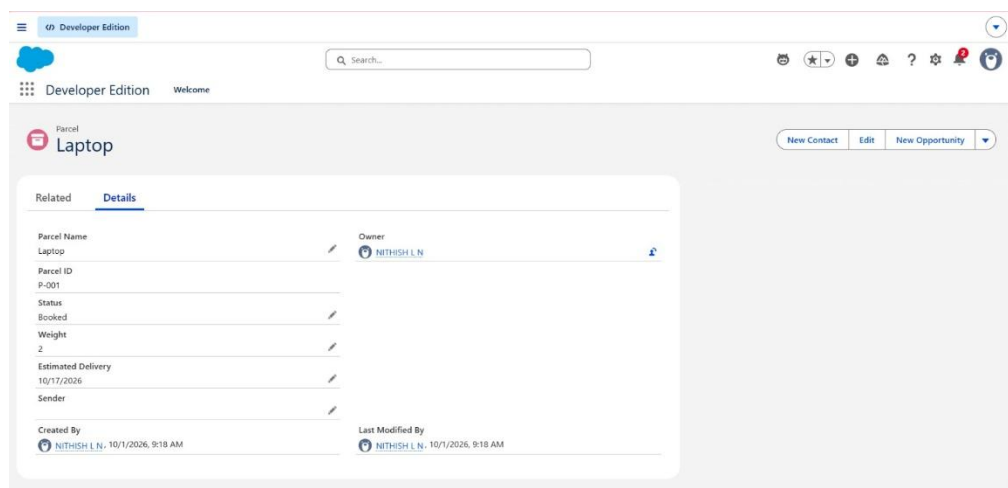


Figure 17. Receiver record page in the Parcel Tracker Lightning App

3.3 Experience Cloud

Experience Cloud is specified as the customer-facing channel, without step-level configuration. Customers would track parcels and see parcel visibility and status through self-service access, with the Customer Profile giving read-only access to their own parcel details and record visibility limited to records linked to their own Sender or Receiver details.

3.4 AgentForce User Experience

A customer's query reaches the SwiftShip Tracker agent, which routes it to the parcel subagent. The subagent runs the Parcel Details Flow, which finds the `Parcel__c` record by `Parcel_ID__c`, calls the `Retrieve_Parcel_Details` prompt and returns a Parcel Tracking Update as the conversational response. If the user gives no Parcel ID, the agent asks for it.

3.5 Reports

The source requires reports for parcel trends, delivery performance and agent activity but gives no report names, filters or screenshots. Four categories follow from the requirements: a parcel status report showing the distribution across Booked, In Transit, Out for Delivery and Delivered; a delivery performance report comparing progress against estimated delivery dates; an agent activity report showing updates by agent; and a parcel flow or operational report showing volume across the lifecycle. All four objects have Allow Reports enabled, so they can serve as report sources.

3.6 Dashboards

Dashboard components are not specified in the source. The reporting requirements are to show parcel status distribution, delivery performance against estimated dates, operational volume, agent activity per agent and shipment trends over time.

PHASE 4: DATA MIGRATION, TESTING & SECURITY SETUP

4.1 Data Migration

The source contains no migration procedure and states no import tool or data volume. Test records, such as the Receiver Sanjana and parcels P-123 and P-002, were created in the Developer org. As a recommendation not documented in the source, sender and receiver data should be cleaned and de-duplicated, parcel statuses should be checked against the picklist, weights and dates validated, and delivery coordinates verified. A load order that follows the relationship dependencies would be Sender, then Parcel, then Receiver, then Delivery, using whichever Salesforce import tool the implementer prefers.

4.2 Security Model

The designed role hierarchy runs from Administrator to Delivery Manager to Agent to Customer. The Admin profile has full access to all objects and records. The Delivery Agent profile has Read and Edit access to Parcel, Delivery, Sender and Receiver. The Customer profile has read-only access to the customer's own parcel details through Experience Cloud, and the Support Staff profile has access to Cases and Parcel details for issue resolution. The source documents no configuration steps for these profiles or the hierarchy, so they are design-level.

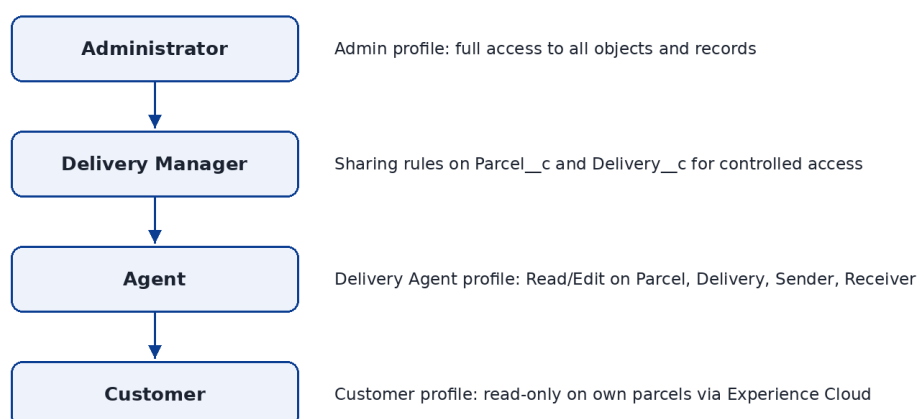


Figure 18. Designed role hierarchy and profile access

4.3 Permission Sets

The `Swift Ship` permission set (API name `Swift_Ship`) uses the Einstein Agent license and is assigned to the Einstein Service Agent user alongside Agentforce Service Agent Secure Base. For each of `Parcel__c`, `Delivery__c`, `Sender__c` and `Receiver__c`, the Read, Create and Edit object permissions are enabled, Delete is optional and left unchecked, and View All, Modify All and View All Fields are unchecked. Field permissions make the key fields visible. The screenshots show Read and Edit access for fields such as Estimated Delivery Date, Sender and Status on `Parcel`; Current Location, Estimated Delivery Date, Parcel and Sender details on `Delivery`; Sender Address and Sender Contact on `Sender`; and Parcel, Receiver's Address, Receiver's Contact and Receiver on `Receiver`. Some fields, such as Parcel ID and the email fields, appear read-only, and the source does not say whether that is deliberate. The set is assigned from Setup, Users, the agent user's Permission Set Assignments, Edit Assignments. Permission sets named `Update Delivery Status` and `Access Agent Console` appear in the design only as examples.

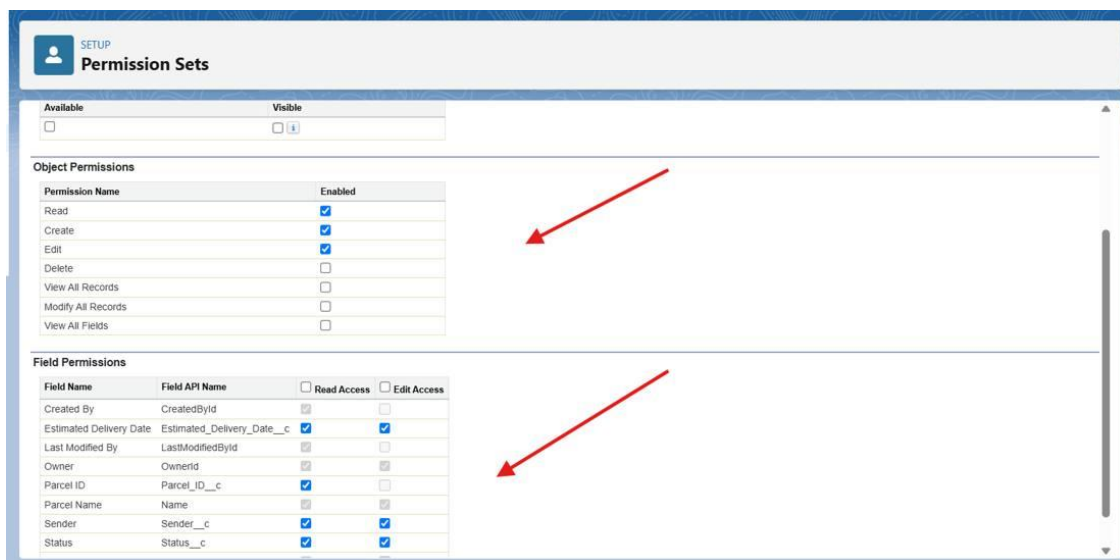


Figure 19. Swift Ship permission set

4.4 Record-Level Security

Sharing rules on `Parcel__c` and `Delivery__c` are designed to give agents and managers controlled access, and customers can see only records linked to their own

Sender or Receiver details, including through Experience Cloud. Organization-wide defaults and sharing rule criteria are not specified.

4.5 Field-Level Security

Sender Email (Sender_Email__c), Receiver Contact (Receiver_Contact__c) and Location Coordinates (Current_Location__c) are identified as sensitive and are restricted by design to the Admin and Agent profiles.

4.6 AgentForce Security

Einstein is turned on with Global Languages and Deploy Prompt Templates, and Agentforce and the Default Agent are enabled. The Einstein Service Agent user, with the Einstein Agent User profile, holds the Swift Ship permission set and Agentforce Service Agent Secure Base. Flow user access is restricted to enabled profiles including Einstein Agent User, and the Flow can run in System Context Without Sharing where data access is restricted. The prompt template is activated, and object access comes from Read, Create and Edit on the four custom objects. Because running without sharing exposes any parcel whose ID is supplied, Flow access and agent topics should be limited to the intended users.

4.7 Testing

The test strategy has twelve cases, and results are stated only where the source supports them; "not evidenced" means the source gives no result, not that the test failed.

TC-01, parcel creation, expects a `Parcel__c` record to save with status, weight, estimated delivery date and sender. It is indirectly evidenced, since parcels P-001 and P-002 appear in the previews, but the creation steps were not captured. TC-02, Parcel ID generation, expects IDs in the form `P-{000}` and is indirectly evidenced by P-001 and P-002 in the preview and final agent output. TC-03, sender and receiver association, is partly evidenced: a Receiver record with a linked Parcel is shown, but the Sender link is not. TC-04, delivery creation, is not evidenced.

TC-05, status update, expects a change to `Status__c` to move a parcel through the lifecycle. It is not evidenced: the configured agent action only retrieves details, and although Figure 16 shows a parcel stored as Out for Delivery, no update was demonstrated. TC-06, current location update, is not evidenced.

TC-07, Prompt Builder retrieval, is evidenced: the preview returned Nithish, P-001, Booked, 10 and 09/29/2026. TC-08, the AgentForce tracking query, is evidenced in two runs: the earlier preview returned P-123, and the final run (Figure 16) took input p-002 through the Parcel Tracker subagent and the Parcel Details action to SandWood, P-002, Out for Delivery, weight 5 and ETA 10/26/2026, evaluated as grounded. TC-09, Flow execution, is evidenced by the Figure 16 trace, which shows the Parcel Details action invoked with `Ids="p-002"` and returning data, although Flow debug results were not captured. TC-10, permission validation, is evidenced indirectly by successful retrieval in both runs, but no negative test was captured. TC-11, customer access through Experience Cloud, and TC-12, reports and dashboards, are not evidenced.

PHASE 5: DEPLOYMENT STRATEGY, MAINTENANCE & DOCUMENTATION

5.1 Deployment Strategy

All configuration was implemented and validated in a standalone Salesforce Developer Edition org for academic and prototype purposes. Production-linked UAT or sandbox deployment was not performed, because Developer Edition orgs do not support production-linked testing, and enterprise AgentForce deployment with sandbox testing, staging and CI/CD automation is outside the project scope. An enterprise path would run Development, Sandbox or Scratch Org, UAT, Staging and Production, migrating custom objects, fields, tabs, Flows, prompt templates, agent configuration and permission sets with change sets, the Salesforce CLI or CI/CD pipelines, after checking Flow versions, permission-set parity, agent and prompt activation and Apex test coverage if Apex is built. The source does not specify these mechanisms; they describe general Salesforce practice.

5.2 AgentForce Deployment

Deployment verification starts in Setup, Agentforce Agents, by confirming the agent appears and is Active. In Agent Builder the parcel subagent is checked for a description related to parcel management and a linked Flow action, and saved and confirmed active. In Setup, Flows, the Flow is confirmed active and handling Parcel ID input, parcel retrieval and the response. The Retrieve Parcel Details template is confirmed active, and the agent is tested in Preview with a query such as "Track parcel ID P12345" or a stored ID such as P-123 or P-002 (Figure 16). The source also proposes testing a status update ("Update delivery status for parcel P67890 to Delivered"), but no update action is documented, so that would need a new Flow-based action. Finally, data is validated in the Parcels tab. The source refers to the Flow as SwiftShip Tracker and the agent as Parcel Tracking Agent in places, whereas the configured Flow is Parcel Details and the agent is SwiftShip Tracker, so names should be aligned.

5.3 Maintenance

Maintenance covers monitoring failed Flow executions and updating Flow logic as processes change, watching for misrouted agent queries and delayed responses, and

refining prompt text, response format and subagent descriptions, scope and instructions. It also covers keeping parcel, customer and delivery data accurate and synchronized, reviewing profiles, permission sets, sharing rules and Flow access, resolving configuration errors, and revisiting Flows and any Apex or batch jobs when requirements change.

5.4 Troubleshooting

Incorrect agent responses usually trace to prompt or topic misconfiguration, so the subagent scope and instructions and the prompt template should be reviewed with the preview's reasoning view. A failed Flow execution should prompt checks that the Flow is active, that `ids` is passed, and a review of Flow debug and failed-interview logs. Incorrect parcel retrieval points to the Get Records filter on `Parcel_ID__c` or an ID format mismatch with `P-{000}`. Permission errors call for confirming the Swift Ship assignment and Flow user access, and data access issues for reviewing field-level security and sharing, with System Context Without Sharing as an option for the Flow. Prompt problems usually come from missing resources or wrong field API names, performance delays from complex Flows or prompt processing, for which the source has no benchmarks, and activation warnings from overlapping subagents, conflicting instructions or Data Cloud not being enabled.

5.5 Documentation

This report is the single source of truth for stakeholders, administrators and developers. It records the data model, object configuration, fields, relationships and security design, the Flow, Prompt Builder and AgentForce configuration, testing evidence and gaps, deployment limits, maintenance and troubleshooting.